



MASTERCAMP — CYBERSÉCURITÉ & MANAGEMENT DE PROJET

PROJET COSTUSSHIELD

Infrastructure de Sécurité Open Source pour PME

(anciennement dénommé OSSI — Open Source Security Infrastructure)

Rapport complet : Cahier des charges, Architecture technique & Bilan de déploiement

Groupe :

Kevin Ojoduma-Quere
Guillaume Ortells
Antoine Poirier
Emmanuel Rajaonera
Mohamed Ouadihi

Table des matières

1	Contexte et cahier des charges	2
1.1	Problématique adressée	2
1.2	Objectif de l'appel d'offres	2
1.3	Contexte de marché	2
1.4	Résultats attendus	2
1.5	Exigences fonctionnelles (méthode MoSCoW)	3
1.6	Méthodologie d'analyse des risques	4
2	Description de la solution envisagée	4
2.1	Vision globale et philosophie architecturale	4
2.2	Périmètre réseau et segmentation	4
2.3	Interconnexion des sites et mobilité des utilisateurs	5
2.4	Analyse active du trafic et observabilité	6
2.5	Intelligence artificielle souveraine et vulgarisation des alertes	6
3	État de l'art et justification des choix technologiques	7
3.1	Positionnement général : open source souverain versus solutions propriétaires	7
3.2	Justification du pare-feu : pfSense face à ses alternatives	7
3.3	Justification du VPN nomade : WireGuard face à OpenVPN	9
3.4	Justification de l'IDS/IPS : Suricata face à Snort	9
3.5	Justification du modèle d'IA : déploiement local via Ollama face aux API cloud propriétaires	10
4	Architecture réellement déployée	11
4.1	Vue d'ensemble de l'infrastructure réalisée	11
4.2	pfSense : le socle confirmé, une segmentation allégée	11
4.3	Tunnel IPSec site-à-site : conformité totale	12
4.4	Accès nomade : OpenVPN à la place de WireGuard	12
4.5	Authentification nomade : certificat et mot de passe, sans TOTP	13
4.6	Suricata : détection active, blocage non activé	13
4.7	Gestion des certificats : autorité interne plutôt qu'ACME	13
4.8	Traffic shaping : conformité confirmée	14
4.9	IA souveraine : brique non implémentée	14
5	Synthèse des écarts et bilan	14
5.1	Tableau de synthèse global	14
5.2	Ce qui a été acquis	14
5.3	Perspectives d'évolution	15
6	Conclusion	15

1 Contexte et cahier des charges

Cette première section reprend les termes de l'appel d'offres à l'origine du projet **CostusShield**, avant toute traduction en choix d'architecture. Elle pose la problématique métier, le contexte de marché et les exigences fonctionnelles qui ont guidé l'ensemble des décisions techniques présentées dans les sections suivantes.

1.1 Problématique adressée

La solution recherchée par le donneur d'ordre vise à répondre à plusieurs problématiques convergentes rencontrées par les petites et moyennes entreprises :

- Le **coût élevé** des solutions commerciales de sécurité des réseaux, souvent hors de portée d'une structure de taille modeste.
- Le besoin de permettre le **télétravail** tout en assurant la sécurité et la confidentialité des communications entre les utilisateurs et les applications de l'entreprise.
- L'**analyse du trafic réseau**, afin de savoir précisément ce qui circule dans le réseau, et de prioriser les applications métier par rapport aux flux de streaming et de navigation Internet non essentiels.

1.2 Objectif de l'appel d'offres

L'appel d'offres a pour objectif de mettre à disposition des petites et moyennes entreprises des solutions permettant de sécuriser leur réseau et leurs communications à **faible coût**, voire **gratuitement**. La solution attendue vise également à faciliter le déploiement de ces technologies et à assurer une **bonne synergie** entre les différents composants mis en œuvre, plutôt qu'un empilement d'outils disparates et mal intégrés.

1.3 Contexte de marché

Les solutions commerciales existantes sur le marché sont très coûteuses et nécessitent une expertise significative pour leur mise en place et leur maintenance, ce qui engendre des coûts supplémentaires difficilement absorbables par une PME. Le projet s'inscrit néanmoins dans la continuité des tendances technologiques actuelles du secteur : *traffic shaping*, systèmes **IDS/IPS**, VPN d'accès à distance, VPN site-à-site, **DNS over TLS**, infrastructures à clé publique (**PKI**), etc. CostusShield ne réinvente donc pas les briques technologiques du marché, mais en propose un assemblage **open source**, cohérent et budgétairement accessible.

1.4 Résultats attendus

Mise à disposition de solutions performantes et abordables — des outils de sécurité réseau fiables, à faible coût voire open source, adaptés aux besoins des petites et moyennes entreprises.

Facilité de déploiement et d'administration — des solutions simples à mettre en œuvre, accompagnées d'une documentation claire, permettant une prise en main rapide sans expertise avancée.

Interopérabilité et intégration fluide — une bonne synergie entre les différents composants (VPN, IDS/IPS, DNS sécurisé, PKI, etc.) pour garantir une architecture cohérente et efficace.

Amélioration de la sécurité globale — réduction des risques liés aux cyberattaques, protection des communications et des données sensibles, conformité aux bonnes pratiques de cybersécurité.

Soutien au télétravail sécurisé — mise en place de solutions d'accès à distance (VPN, chiffrement, authentification) garantissant la confidentialité des échanges.

Réduction des coûts — grâce à des solutions pérennes, évolutives, et à une réduction des besoins en support externe.

1.5 Exigences fonctionnelles (méthode MoSCoW)

Les exigences fonctionnelles de l'appel d'offres ont été hiérarchisées selon la méthode **MoSCoW** (*Must have*, *Should have*, *Could have*), permettant d'arbitrer les priorités de développement dans un contexte de délai contraint.

Must Have (Indispensable)

- Sécurisation du réseau local et des communications (chiffrement, pare-feu, VPN site-à-site et accès distant).
- Intégration d'un système IDS/IPS pour la détection et la prévention des intrusions.
- Interface de gestion centralisée, simple d'utilisation.
- Compatibilité avec les standards de sécurité (PKI, DNS over TLS, etc.).
- Documentation complète pour le déploiement et la maintenance.
- Solution à coût réduit, adaptée aux PME.

Should Have (Fortement recommandé)

- Fonctionnalité de *traffic shaping* pour prioriser les flux critiques (applications métier).
- Mises à jour automatiques ou faciles à appliquer (sécurité, fonctionnalités).
- Journalisation des événements (logs de sécurité, surveillance du trafic).
- Authentification forte pour l'accès distant (2FA, certificats).
- Support multi-plateformes (Linux, Windows, etc.).

Could Have (Optionnel mais utile)

- Tableau de bord de supervision en temps réel.
- Intégration possible avec des outils externes (SIEM, annuaire LDAP).
- Fonctionnalité de sauvegarde/restauration de la configuration.
- Interface multilingue.

1.6 Méthodologie d'analyse des risques

En parallèle de la hiérarchisation fonctionnelle, une démarche d'analyse de risque a encadré la définition du périmètre de sécurité à couvrir :

1. **Identification des actifs** : recenser les ressources matérielles, logicielles, humaines, process et données critiques.
2. **Identification des menaces et vulnérabilités** : lister ce qui pourrait affecter chaque actif (cyberattaques, pannes, erreurs humaines, catastrophes).
3. **Analyse du risque** : estimer la probabilité et l'impact de chaque scénario de menace selon une échelle qualitative.
4. **Évaluation du risque** : comparer les niveaux de risque obtenus à des critères d'acceptabilité pour prioriser les traitements.
5. **Traitement du risque** : sélectionner et appliquer des mesures de sécurité (prévention, détection, correction, transfert).

Cette grille de lecture, inspirée des référentiels d'analyse de risque usuels (type EBIOS), a directement nourri les choix d'architecture détaillés dans la section suivante.

2 Description de la solution envisagée

2.1 Vision globale et philosophie architecturale

La solution **CostusShield** que nous proposons repose sur une philosophie de **défense en profondeur** (*Defense in Depth*), principe fondateur de la sécurité des systèmes d'information qui consiste à multiplier les couches de protection indépendantes afin qu'aucun point de défaillance unique ne compromette l'intégrité globale du système. Contrairement aux architectures monolithiques où la sécurité est concentrée sur un seul équipement périmétrique, l'approche retenue distribue les mécanismes de contrôle à chaque niveau de la pile réseau : filtrage stateful au niveau du pare-feu, inspection applicative au niveau de l'IDS/IPS, cloisonnement logique par la segmentation, et surveillance comportementale par la collecte centralisée des journaux.

L'ensemble des briques technologiques composant cette architecture est issu de l'écosystème **open source**, garantissant ainsi la **souveraineté numérique** de la PME concernée. Cette souveraineté se traduit concrètement par l'absence de dépendance vis-à-vis d'éditeurs propriétaires, par la maîtrise totale du code déployé, et par le maintien de l'ensemble des données sensibles au sein du périmètre de confiance de l'entreprise. La solution est dimensionnée pour couvrir trois types d'entités : un **site principal** hébergeant les serveurs et les ressources critiques, un **site isolé** dont les flux doivent être intégrés de manière sécurisée, et une population de **télétravailleurs** nécessitant un accès nomade fiable et chiffré.

2.2 Périmètre réseau et segmentation

La pierre angulaire de toute architecture sécurisée réside dans la **segmentation réseau**, qui consiste à subdiviser l'espace d'adressage en zones logiques étanches, chacune soumise

à une politique de filtrage adaptée à son niveau de confiance et à la criticité des ressources qu'elle héberge. La solution CostusShield articule l'infrastructure autour de quatre zones distinctes.

La **zone WAN** (*Wide Area Network*) constitue l'interface entre le système d'information de la PME et les réseaux non maîtrisés, notamment Internet. Tout flux entrant en provenance de cette zone est présumé hostile jusqu'à preuve du contraire et est soumis à un filtrage strict par le pare-feu **pfSense**, qui opère comme première ligne de défense.

La **zone LAN** (*Local Area Network*) regroupe l'ensemble des postes de travail, des équipements utilisateurs et des serveurs internes. Cette zone bénéficie d'un niveau de confiance supérieur, mais reste soumise à des règles de filtrage interne afin de contenir les mouvements latéraux qui pourraient résulter d'une compromission d'un poste.

La **zone DMZ** (*Demilitarized Zone*) héberge les services exposés vers l'extérieur — serveurs web, relais de messagerie, points de terminaison VPN — qui nécessitent d'être accessibles depuis Internet tout en étant strictement isolés du LAN. Un serveur compromis en DMZ ne doit en aucun cas constituer un pivot vers le réseau interne.

La **zone VPN** regroupe les tunnels chiffrés, tant site-à-site qu'itinérants, et constitue une zone de transit contrôlée permettant d'étendre le périmètre de confiance aux entités distantes de manière maîtrisée. Les règles de filtrage appliquées à cette zone différencient explicitement les flux légitimes des connexions anormales.

Cette segmentation rigoureuse réduit mécaniquement la **surface d'attaque** de la PME en limitant la portée de toute intrusion à la zone compromise, et en rendant coûteuse toute tentative de mouvement latéral vers des zones à plus haute criticité.

Le degré de fidélité de cette segmentation par rapport à ce qui a réellement été implémenté est détaillé à la section 4.

2.3 Interconnexion des sites et mobilité des utilisateurs

a) Tunnel IPSec site-à-site

L'interconnexion permanente entre le site principal et le site isolé est assurée par un **tunnel IPSec** (*Internet Protocol Security*) en mode *site-à-site*. Ce protocole de sécurité opère au niveau de la couche réseau (couche 3 du modèle OSI) et garantit la **confidentialité**, l'**intégrité** et l'**authenticité** des communications inter-sites en s'appuyant sur des algorithmes de chiffrement éprouvés, comme **AES-256** pour le chiffrement des données et **SHA-256** ou **SHA-512** pour l'intégrité, ainsi que sur le protocole **IKEv2** (*Internet Key Exchange version 2*) pour la négociation automatisée des associations de sécurité.

Le tunnel IPSec est configuré en mode *always-on*, c'est-à-dire qu'il est automatiquement rétabli en cas d'interruption de la connexion WAN, assurant ainsi la robustesse et la permanence de l'interconnexion. Cette approche est nativement prise en charge par **pfSense**, qui expose une interface de configuration graphique permettant de paramétrer les politiques de phase 1 (IKE) et de phase 2 (ESP/AH) avec précision.

b) VPN nomade pour les télétravailleurs

L'accès sécurisé des télétravailleurs est assuré par **WireGuard**, un protocole VPN fondé sur des primitives cryptographiques de nouvelle génération (**ChaCha20** pour le chiffrement, **Poly1305** pour l'authentification, **Curve25519** pour l'échange de clés). WireGuard est intégré directement à pfSense sous la forme d'un module natif, simplifiant le déploiement et la gestion des pairs.

L'**authentification forte** est assurée par la combinaison d'une **infrastructure à clés publiques (PKI)** dédiée, qui permet la génération, la signature et la révocation de certificats clients, et d'un mécanisme d'**authentification à deux facteurs (2FA)** s'appuyant sur le protocole **TOTP** (*Time-based One-Time Password*). Cette combinaison garantit qu'un certificat seul, en cas de vol, ne suffit pas à établir une connexion illégitime. OpenVPN est conservé en solution de repli afin d'assurer la compatibilité avec les équipements ou les environnements réseau qui ne supportent pas WireGuard.

Comme détaillé à la section 4, c'est finalement OpenVPN qui a été déployé en production à la place de WireGuard.

2.4 Analyse active du trafic et observabilité

La visibilité sur les flux traversant l'infrastructure est une condition nécessaire à toute posture de sécurité proactive. Le couplage entre la **détection et prévention d'intrusions** (IDS/IPS) et la **collecte centralisée des journaux** constitue le dispositif d'observabilité de la solution CostusShield.

Suricata est déployé en mode **IPS** (*Inline Prevention System*) en coupure sur les interfaces réseau exposées du pare-feu pfSense. Il opère une inspection approfondie des paquets (**DPI** — *Deep Packet Inspection*), confrontant le trafic à des jeux de règles régulièrement mis à jour notamment les ensembles **Emerging Threats** et **Snort VRT**, afin d'identifier les signatures d'attaques connues, les comportements anormaux et les tentatives d'exploitation de vulnérabilités. En mode IPS, Suricata dispose du pouvoir de **bloquer en temps réel** les paquets identifiés comme malveillants, sans intervention humaine, réduisant ainsi drastiquement le temps de réaction face aux menaces courantes.

La **collecte et la centralisation des journaux** est assurée par un agent **syslog** configuré sur pfSense et Suricata, qui remonte l'ensemble des événements vers une instance centralisée. L'analyse corrélée de ces flux de logs permet d'identifier des schémas d'attaque distribués qui seraient imperceptibles en analysant chaque source de manière isolée.

2.5 Intelligence artificielle souveraine et vulgarisation des alertes

L'un des défis majeurs de la cybersécurité en contexte PME réside dans la **fracture sémantique** qui sépare les alertes brutes générées par les outils de sécurité de la capacité de compréhension des décideurs non techniciens. Une alerte Suricata telle que **ET SCAN Nmap Scripting Engine User-Agent Detected** ou un log pfSense mentionnant des paquets bloqués avec des flags TCP anormaux ne permet pas à un dirigeant d'entreprise d'évaluer le risque réel et d'arbitrer une réponse appropriée.

La solution CostusShield envisageait d'intégrer un **modèle de langage de grande taille** (**LLM** — *Large Language Model*) déployé localement via le moteur d'inférence **Olama**. Ce LLM devait ingérer quotidiennement les alertes brutes collectées par Suricata ainsi que les événements significatifs remontés par pfSense, et générer automatiquement un **Rapport No-Expert** : un document synthétique, rédigé en langage naturel et compréhensible sans formation technique, qui résume les incidents de la journée, leur niveau de gravité, les actions correctives déjà prises automatiquement et, le cas échéant, les points d'attention nécessitant une décision humaine.

La nature **locale** du déploiement envisagé était fondamentale : les journaux de sécurité, qui peuvent contenir des informations stratégiquement sensibles (tentatives de connexion, noms d'utilisateurs, adresses IP internes, communications métier), ne devaient **jamais**

quitter le périmètre de confiance de l'entreprise — à l'inverse d'une utilisation d'API cloud propriétaires (OpenAI, Anthropic et autres), structurellement incompatible avec les exigences de conformité au RGPD dans ce contexte.

Cette brique a fait l'objet d'un arbitrage de gestion de projet explicité à la section 4 : elle n'a pas été implémentée.

3 État de l'art et justification des choix technologiques

3.1 Positionnement général : open source souverain versus solutions propriétaires

Le marché de la sécurité réseau pour les PME est dominé par des acteurs propriétaires dont les offres, bien que fonctionnellement complètes, imposent des modèles économiques et des contraintes de dépendance incompatibles avec les contraintes budgétaires et les exigences de souveraineté des petites et moyennes entreprises. Le tableau 1 établit une comparaison structurée entre l'approche retenue pour CostusShield et les offres représentatives du marché propriétaire.

Il ressort de cette comparaison que les solutions propriétaires offrent un avantage substantiel en matière de support contractualisé, ce qui peut constituer un facteur décisif pour des entreprises dépourvues de compétences internes. Toutefois, pour une PME cherchant à maîtriser ses coûts et à préserver sa souveraineté numérique, l'approche open source, complétée par un contrat de prestation avec un intégrateur spécialisé si nécessaire, présente un rapport valeur/coût nettement supérieur.

3.2 Justification du pare-feu : pfSense face à ses alternatives

Le choix du composant pare-feu détermine en grande partie la robustesse et l'évolutivité de toute l'architecture. L'écosystème open source propose plusieurs distributions spécialisées dont les trois principales sont **pfSense**, **OPNsense** et **IPFire**.

IPFire est une distribution Linux orientée pare-feu dont la conception simple la rend accessible mais limite sa capacité à servir de socle à une architecture complexe. Son écosystème de modules additionnels (*Pakfire*) est sensiblement plus restreint que celui de ses concurrents, et sa communauté, bien qu'active, demeure de taille plus modeste. Pour un déploiement intégrant simultanément IPSec, WireGuard, Suricata et du *traffic shaping* avancé, IPFire impose une complexité de configuration manuelle supérieure sans apporter de bénéfice différenciant.

OPNsense est un fork de pfSense apparu en 2015, dont la qualité de l'interface graphique et le modèle de mises à jour plus fréquent sont généralement reconnus. Il présente toutefois un historique de déploiement en production moins étendu que pfSense, et son écosystème de plugins tiers est encore en cours de maturation sur certains points.

pfSense est retenu comme socle pare-feu de la solution CostusShield pour les raisons suivantes. D'abord, sa **maturité** : développé depuis 2004 sur la base du système d'exploitation **FreeBSD**, il bénéficie d'une base d'utilisateurs et de déploiements en production considérable, d'une documentation exhaustive et d'un historique de correctifs de sécurité rigoureux. Ensuite, son **écosystème de packages** : le gestionnaire de paquets intégré propose nativement Suricata, WireGuard, OpenVPN, *ntopng*, ainsi qu'un gestionnaire de certificats PKI (**ACME/Let's Encrypt**) et des modules de *traffic shaping* basés sur **HFSC**

Table 1: Comparaison générale : approche open source (CostusShield) vs solutions propriétaires

Critère	CostusShield (Open Source)	Fortinet FortiGate	Cisco ASA / Firepower
CAPEX initial	Matériel générique uniquement ; licences logicielles nulles	Acquisition du boîtier propriétaire requise (3 000 à 20 000 € selon gamme)	Acquisition du boîtier propriétaire requise (5 000 à 30 000 € selon gamme)
OPEX annuel	Maintenance communautaire gratuite ; coût limité à l'hébergement et au temps administrateur	Abonnement FortiCare et FortiGuard obligatoire (1 500 à 8 000 €/an)	Abonnements SmartNet et Talos Intelligence obligatoires (2 000 à 10 000 €/an)
Souveraineté et contrôle des données	Total : code auditable, données confinées localement, absence de télémétrie obligatoire	Partielle : télémétrie vers les serveurs Fortinet, résidence des données non garantie en Europe	Partielle : télémétrie vers les serveurs Cisco Talos, dépendance aux infrastructures cloud américaines
Flexibilité et personnalisation	Maximale : accès au code source, écosystème de plugins utile, adaptation à toute topologie	Limitée : personnalisation contrainte par les fonctionnalités du firmware propriétaire	Limitée : personnalisation contrainte par les licences et les versions de l'IOS/ASA OS
Support et garantie	Communautaire (forums, documentation, prestataires tiers) ; pas de SLA garanti par défaut	Support constructeur avec SLA contractuel (réponse en 4h à 24h selon niveau)	Support constructeur avec SLA contractuel (TAC Cisco, 24h/24 7j/7 selon contrat)

et **PRIQ**. Enfin, sa **large communauté** garantit la pérennité des développements et la disponibilité de ressources d'expertise sur le marché des prestataires.

3.3 Justification du VPN nomade : WireGuard face à OpenVPN

L'accès VPN nomade impose des exigences spécifiques souvent négligées : faible latence, robustesse lors des changements de réseau (itinérance entre Wi-Fi et données mobiles), et facilité de déploiement sur des équipements hétérogènes (Windows, macOS, Android, iOS). Cette comparaison porte sur les deux solutions open source de référence que sont **WireGuard** et **OpenVPN**.

OpenVPN, créé en 2001, est la référence historique du VPN open source. Il s'appuie sur la bibliothèque **OpenSSL** et supporte une très large gamme de chiffrements et de configurations, ce qui en fait une solution universelle mais également complexe. Sa base de code dépasse 70 000 lignes, ce qui rend son audit de sécurité exhaustif particulièrement laborieux. Sur le plan des performances, OpenVPN opère en espace utilisateur, ce qui engendre une surcharge notable due aux multiples changements de contexte entre l'espace noyau et l'espace utilisateur lors du traitement de chaque paquet.

WireGuard, publié en 2016 et intégré au noyau Linux depuis la version 5.6 (mars 2020), adopte une philosophie radicalement différente. Sa base de code ne dépasse pas 4 000 lignes, facilitant considérablement l'audit de sécurité et réduisant mécaniquement la surface d'attaque cryptographique. Il s'appuie sur des **primitives cryptographiques modernes** et soigneusement choisies : **ChaCha20-Poly1305** pour le chiffrement authentifié, **Curve25519** pour l'échange de clés *Diffie-Hellman*, et **BLAKE2s** pour le hachage. L'exécution en espace noyau (*kernel space*) élimine la surcharge des changements de contexte et se traduit par des **performances brutes** nettement supérieures à OpenVPN, avec une **latence réduite** et un débit significativement plus élevé, notamment sur les connexions mobiles. La gestion de l'**itinérance réseau** est native : WireGuard maintient le tunnel lors d'un changement d'adresse IP sans nécessiter de reconnexion, contrairement à OpenVPN qui doit renégocier la session.

WireGuard est donc privilégié comme solution principale pour les accès nomades de la solution CostusShield. OpenVPN est conservé en **solution de repli** pour les environnements réseau restrictifs (réseaux d'hôtels ou d'entreprises tiers) qui bloquent le port UDP utilisé par WireGuard, OpenVPN pouvant être configuré sur le port TCP 443 pour contourner ces restrictions.

3.4 Justification de l'IDS/IPS : Suricata face à Snort

La détection et la prévention d'intrusions constituent le mécanisme d'analyse active le plus exigeant en termes de ressources de traitement, puisqu'elles nécessitent l'inspection en temps réel de l'intégralité des paquets traversant les interfaces surveillées. Le comparatif porte ici sur les deux moteurs open source historiques : **Suricata** et **Snort**.

Snort, créé par Sourcefire en 1998 et aujourd'hui maintenu par **Cisco Talos**, est le pionnier des IDS open source. Son modèle d'exécution repose sur une **architecture monothreadée** : un seul fil d'exécution traite l'ensemble des paquets capturés, ce qui constituait une conception adaptée aux débits réseau des années 2000, mais qui représente aujourd'hui un **goulot d'étranglement** structurel face aux infrastructures 10 Gbps et au-delà. Si Snort 3 (publié en 2021) introduit des améliorations significatives, y compris un support limité du multithreading, son architecture native reste fondamentalement

différente de celle de Suricata.

Suricata, développé par l'**OISF** (*Open Information Security Foundation*) depuis 2009, a été conçu dès l'origine avec une **architecture multithreadée native**. L'ensemble des opérations (capture des paquets, décodage, détection de signatures, journalisation) est distribué sur l'ensemble des cœurs processeur disponibles, permettant une mise à l'échelle quasi linéaire des performances avec le nombre de cœurs. Cette conception le rend intrinsèquement supérieur à Snort pour le traitement des **flux à haut débit** typiques des infrastructures réseau modernes.

Par ailleurs, Suricata supporte nativement des fonctionnalités avancées indispensables à la solution CostusShield : l'**inspection TLS/SSL** (analyse des métadonnées des flux chiffrés sans déchiffrement), la **détection de protocoles applicatifs** par inspection comportementale (*protocol detection*), la **journalisation structurée en JSON** via le format **EVE** (*Extensible Event Format*) qui facilite l'intégration avec des outils d'analyse aval, ainsi qu'une compatibilité totale avec les jeux de règles **Snort VRT** et **Emerging Threats Open**, assurant la portabilité des politiques de détection existantes.

Suricata est donc retenu comme moteur IDS/IPS de la solution CostusShield, en s'appuyant sur les jeux de règles **Emerging Threats** maintenus par **Proofpoint**, régulièrement mis à jour et couvrant les menaces les plus récentes.

3.5 Justification du modèle d'IA : déploiement local via Ollama face aux API cloud propriétaires

L'intégration de l'intelligence artificielle générative dans une chaîne de traitement de données de sécurité soulève une question architecturale fondamentale : le modèle doit-il être hébergé localement ou interrogé via une **API cloud** propriétaire ? L'analyse comparative présentée ici oppose le déploiement local via **Ollama** aux services d'API offerts par les principaux acteurs du marché, notamment **OpenAI GPT-4** et les offres cloud de fournisseurs tiers.

L'utilisation d'une API cloud propriétaire présente l'avantage d'accéder à des modèles de très grande taille (*frontier models*) sans infrastructure dédiée. Cependant, cette approche est structurellement incompatible avec le contexte de la sécurité réseau pour plusieurs raisons fondamentales.

Premièrement, la **confidentialité des données** : les journaux générés par Suricata et pfSense contiennent des informations hautement sensibles : adresses IP internes, noms de domaines accédés, comptes utilisateurs impliqués dans des événements de sécurité, topologie interne du réseau. L'envoi de ces données vers des serveurs tiers situés hors du périmètre de l'entreprise, *a fortiori* vers des infrastructures hébergées en dehors de l'Union Européenne, constitue une **violation potentielle du Règlement Général sur la Protection des Données (RGPD)**, notamment au regard de l'article 46 relatif aux transferts de données vers des pays tiers et de l'article 32 relatif à la sécurité du traitement. Le risque de **fuite de données sensibles** vers des tiers non contrôlés est donc structurel et non mitigeable dans ce modèle.

Deuxièmement, le **coût opérationnel** : les API cloud propriétaires facturent à l'usage, indexé sur le volume de tokens traités. Dans un contexte de journalisation continue où plusieurs milliers d'événements sont générés quotidiennement, les coûts d'inférence peuvent atteindre des montants significatifs à l'échelle d'un déploiement annuel.

Troisièmement, la **dépendance** : toute interruption du service API cloud entraîne l'indisponibilité complète de la fonctionnalité de rapport automatisé, sans recours possible.

Ollama est un moteur d'inférence local qui permet d'exécuter des modèles de langage optimisés (famille **Llama 3**, **Mistral**, **Qwen2** et autres) sur du matériel générique, y compris sans **GPU** dédié pour des modèles quantifiés de taille modeste (7B à 13B de paramètres). Déployé sur un serveur hébergé dans les locaux de la PME ou dans son environnement virtualisé privé, il garantit que **l'ensemble du cycle de traitement** (ingestion des logs, inférence, génération du rapport) s'effectue exclusivement au sein du périmètre de confiance de l'entreprise.

L'IA souveraine via Ollama était donc retenue, sur le papier, comme composant intégral de la solution CostusShield. Les raisons pour lesquelles cette brique n'a finalement pas été implémentée sont détaillées à la section 4.

4 Architecture réellement déployée

Cette section présente l'infrastructure telle qu'elle a été effectivement configurée et validée, sur la base de l'export de configuration pfSense et du profil de connexion client distribué aux télétravailleurs. Chaque sous-section confronte le choix initialement prévu (section 2) à l'implémentation réelle, dans un objectif de transparence méthodologique.

Note technique : les artefacts de configuration (autorité de certification, noms d'interfaces, noms de domaine internes) portent encore les identifiants générés durant la phase de déploiement, antérieure au renommage du projet en CostusShield. Ils sont reproduits ici sans modification, par souci de fidélité documentaire.

4.1 Vue d'ensemble de l'infrastructure réalisée

L'infrastructure repose sur une instance **pfSense 23.3** dotée de trois interfaces physiques :

- **WAN** (em0) — 192.168.100.1/24
- **LAN** (em1) — 10.10.10.254/24, plage DHCP .50 à .100
- **OPT1** (em2) — client DHCP, sans règle de filtrage spécifique

Le site distant est joint via un tunnel IPSec vers la passerelle 192.168.100.2, exposant le réseau 10.20.20.0/24. L'accès nomade des télétravailleurs transite par un serveur OpenVPN sur le port UDP/1194, avec un réseau de tunnel dédié 10.50.50.0/24.

4.2 pfSense : le socle confirmé, une segmentation allégée

Écart partiel. pfSense se confirme comme un socle solide : la maturité, l'écosystème de paquets et la stabilité annoncés en section 3.2 se vérifient à l'usage. En revanche, la segmentation en quatre zones logiques (WAN / LAN / DMZ / VPN) décrite en section 2.2 a été simplifiée : seules trois interfaces physiques existent, une règle **Default allow LAN to any** reste active, et l'interface OPT1 — pressentie comme zone DMZ — ne dispose d'aucune règle de filtrage dédiée au-delà de la mise en forme du trafic. Faute de temps pour affiner les règles inter-zones, l'isolement strict d'une DMZ n'a donc pas été formalisé à ce stade.

4.3 Tunnel IPSec site-à-site : conformité totale

Conforme. Le tunnel IPSec est la brique la plus fidèle au cahier des charges initial. Les paramètres relevés dans la configuration correspondent trait pour trait à ceux annoncés en section 2.3 :

Table 2: Paramètres IPSec relevés dans la configuration déployée

Paramètre	Valeur relevée
Version IKE	IKEv2
Chiffrement (phase 1 et 2)	AES-256
Hash / intégrité	SHA-256
Groupe Diffie-Hellman	Groupe 14 (2048 bits)
Authentification	Clé pré-partagée (PSK)
Mode	Tunnel
PFS (<i>Perfect Forward Secrecy</i>)	Groupe 14
Durée de vie phase 1 / phase 2	28 800 s / 3 600 s
Détection de perte de lien (DPD)	Intervalle 10 s, 5 échecs tolérés
NAT-Traversal	Activé
Réseau distant	10.20.20.0/24
Passerelle distante	192.168.100.2

4.4 Accès nomade : OpenVPN à la place de WireGuard

Technologie remplacée. Contrairement à ce qui était prévu en section 3.3, **aucune instance WireGuard** n'est active dans la configuration pfSense. C'est **OpenVPN** qui assure, seul, l'accès nomade des télétravailleurs.

Table 3: Paramètres OpenVPN relevés dans la configuration déployée

Paramètre	Valeur relevée
Protocole / port	UDP 1194
Mode d'interface	tun (niveau 3, routage IP)
Ciphers de données	AES-256-GCM, AES-128-GCM, ChaCha20-Poly1305
Repli si négociation échoue	AES-256-CBC
Digest d'intégrité	SHA-256
Taille de clé Diffie-Hellman	2048 bits
Couche additionnelle	TLS-Auth (signature HMAC des paquets de contrôle)
Réseau du tunnel	10.50.50.0/24
Réseau local exposé	10.10.10.0/24
Clients simultanés maximum	5

Raison identifiée. Le module WireGuard pour pfSense s'est révélé instable dans le temps imparti au projet. OpenVPN, plus universel et mieux éprouvé sur cette version de pfSense, a permis de livrer un accès nomade fonctionnel et chiffré, au prix d'une latence et d'une simplicité de configuration moindres que ce qui était initialement visé.

4.5 Authentification nomade : certificat et mot de passe, sans TOTP

Fonctionnalité non implémentée. La PKI existe bel et bien : une autorité de certification interne auto-signée (OSSI-ROOT-CA) a signé un certificat serveur ainsi qu'un certificat client individuel (`teleworker-cert`). Le profil client OpenVPN confirme une vérification stricte du certificat serveur (`remote-cert-tls server, verify-x509-name vpn.ossi.local`).

En complément du certificat, une authentification par **identifiant / mot de passe local** est exigée (directive `auth-user-pass`). En revanche, **aucun module TOTP** n'a été installé ou configuré sur l'instance pfSense : la double authentification temporelle décrite en section 2.3 n'a pas été mise en œuvre. L'authentification repose donc sur deux facteurs de nature proche (« quelque chose que l'on possède » — le certificat — et « quelque chose que l'on sait » — le mot de passe), plutôt que sur un véritable second facteur temporel indépendant.

4.6 Suricata : détection active, blocage non activé

Mode réduit. Suricata 7.0.8 est bien actif sur les interfaces **WAN** et **LAN** (règles nommées `IDS-Entreprise-WAN` et `IDS-Entreprise-LAN` dans la configuration). Le jeu de règles **Emerging Threats Open** est activé et à jour ; les rulesets Snort VRT et ET Pro (payant) restent désactivés, conformément à une utilisation gratuite du moteur.

Le point de vigilance porte sur le mode de fonctionnement : le paramètre `blockoffenders` est positionné sur `off` pour les deux interfaces surveillées, et la politique IPS (`ips_policy_enable`) n'est pas activée. Contrairement à ce qui était annoncé en section 2.4, Suricata fonctionne donc en mode **IDS** (détection et journalisation des alertes) et non en mode **IPS actif** (blocage automatique en coupure). La visibilité sur les menaces réseau est bien réelle ; l'automatisation de la réponse ne l'est pas encore.

4.7 Gestion des certificats : autorité interne plutôt qu'ACME

Approche adaptée. Le rapport initial mentionnait une intégration **ACME / Let's Encrypt** pour la gestion des certificats. En pratique, une autorité de certification interne auto-signée (OSSI-ROOT-CA) a été mise en place, avec émission manuelle des certificats serveur et client.

Ce choix est cohérent avec le contexte : Let's Encrypt délivre des certificats pour des noms de domaine publiquement résolubles, ce qui ne correspond pas à l'usage d'un VPN interne (`vpn.ossi.local` n'étant pas un nom public). Une CA interne est l'approche standard dans ce cas de figure — il ne s'agit donc pas d'un abandon technique, mais d'une adaptation pertinente au périmètre réel du projet.

4.8 Traffic shaping : conformité confirmée

Conforme. Les files d'attente **HFSC** (sur les interfaces LAN et OPT1) et **PRIQ** (sur l'interface WAN) sont actives, avec une bande passante configurée jusqu'à 100 Mb/s selon les interfaces. La notification explicite de congestion (**ECN**) est activée, permettant de signaler la saturation d'une file sans nécessairement supprimer de paquets. Cette brique correspond fidèlement à ce qui était annoncé en section 3.2.

4.9 IA souveraine : brique non implémentée

Abandonné. Aucune trace d'intégration Ollama ou de tout autre moteur d'inférence local n'apparaît dans la configuration déployée. En accord avec une démarche de gestion des risques et de respect des délais, la brique IA décrite en section 2.5 a été écartée pour prioriser la robustesse et la mise en production de l'infrastructure réseau principale. Il s'agit d'un arbitrage de gestion de projet assumé, et non d'un échec technique : consolider ce qui protège directement l'entreprise avant d'ajouter une fonctionnalité de confort.

Synthèse des écarts et bilan

5.1 Tableau de synthèse global

Le tableau 4 consolide, composant par composant, la comparaison entre ce qui était prévu dans le cahier des charges technique et ce qui a été effectivement réalisé.

Table 4: Synthèse globale : prévu versus réalisé

Composant	Prévu (rapport)	Réalisé	Statut
Pare-feu	pfSense, 4 zones	pfSense, 3 interfaces	Partiel
Interconnexion sites	IPSec IKEv2 AES-256	IPSec IKEv2 AES-256	Conforme
VPN nomade	WireGuard	OpenVPN	Remplacé
Authentification VPN	Certificat + TOTP	Certificat + mot de passe	Partiel
PKI	ACME / Let's Encrypt	CA interne auto-signée	Adapté
IDS/IPS	Suricata en blocage actif	Suricata en détection (IDS)	Partiel
Traffic shaping	HFSC / PRIQ	HFSC / PRIQ	Conforme
IA souveraine	LLM local via Ollama	Non implémenté	Abandonné

5.2 Ce qui a été acquis

- Une infrastructure réseau fonctionnelle de bout en bout, couvrant site principal, site distant et télétravailleurs.

- Un tunnel site-à-site strictement conforme aux standards annoncés.
- Une visibilité réelle sur les menaces réseau, grâce à Suricata en mode détection sur les interfaces critiques.
- Une base saine, documentée et évolutive, sur laquelle les ajustements identifiés ci-dessous peuvent être appliqués sans refonte complète.

5.3 Perspectives d'évolution

1. Activer le **blocage actif** (**blockoffenders**) sur Suricata pour passer d'un mode IDS à un véritable mode IPS.
2. Réévaluer l'intégration de **WireGuard** sur une version de pfSense plus récente ou via un correctif du module concerné.
3. Ajouter un **second facteur TOTP** réel, indépendant du certificat et du mot de passe.
4. Définir des règles de filtrage dédiées pour l'interface **OPT1**, afin de formaliser une véritable zone DMZ isolée.
5. Durcir la règle LAN **vers any** par défaut, en la remplaçant par des règles explicites par flux métier.

6 Conclusion

Le projet **CostusShield** tient la promesse centrale de l'appel d'offres : une infrastructure réseau segmentée, interconnectée et surveillée, construite exclusivement à partir de briques open source, sans licence propriétaire. La comparaison point par point entre le rapport d'architecture initial et la configuration réellement déployée révèle des écarts réels, mais circonscrits à des ajustements de temps et de robustesse — non à un abandon de la vision du projet. Le tunnel IPSec site-à-site et le *traffic shaping* respectent scrupuleusement le cahier des charges technique ; l'accès nomade, l'authentification forte et le passage en blocage actif de Suricata constituent les chantiers prioritaires pour amener CostusShield au niveau de maturité initialement visé.